

RNA Secondary Structure Prediction via QUBO Formulation and Quantum Optimization

WISER × Moderna Quantum Challenge — Technical Report

*Classical benchmarking (ViennaRNA) and quantum/quantum-inspired methods
(QUBO encoding + QAOA / CVaR-VQE in PennyLane) for minimum free energy folding*

August 2026

Submitted by: Yagna Priya Gummadi, Sri Jahnavi Chinthalapudi, Sree Neha
WISER Global Quantum+AI Program 2026

1. Executive Summary

This project reframes RNA secondary-structure prediction — normally solved by classical dynamic programming (ViennaRNA / Zuker's algorithm) — as a discrete combinatorial optimization problem, and hands that problem to quantum optimization circuits. Each candidate base pair becomes a binary decision variable; Watson-Crick pairing, wobble (G-U) pairing, a minimum hairpin-loop constraint, and penalties for overlapping or crossing (pseudoknotted) pairs are all folded into a single **QUBO** (Quadratic Unconstrained Binary Optimization) objective. Two independent quantum methods — **QAOA** and **CVaR-VQE**, both implemented in PennyLane — solve this QUBO and are checked against a classical ViennaRNA ground truth, not just each other.

The headline result: across a curated 12-sequence benchmark (8–14 nt, 33%–100% GC content), CVaR-VQE reproduces its own QUBO's exact optimum on **every sequence tested (12/12 trial batches, success rate 1.0)**, and that optimum matches the real ViennaRNA minimum-free-energy (MFE) structure exactly — basepair F1 = 1.0, real thermodynamic energy gap = 0.0 kcal/mol — on **11 of 12 sequences**. QAOA is markedly less reliable under the same step budget, with a mean success rate of only **~48%** across the sequences it was run on, and is capped below ~11–13 qubits by both accuracy and runtime. Structural comparison uses ViennaRNA's own thermodynamic evaluator (same units as the MFE, not the arbitrary internal QUBO score), so the reported agreement is a real physical comparison, not an artifact of the optimizer matching its own objective.

Headline numbers at a glance

CVaR-VQE: 100% QUBO-optimum success rate across all tested sequences; exact ViennaRNA MFE match (F1 = 1.0) on 11/12 curated sequences.

QAOA: ~48% mean success rate under the same optimizer budget; reliable only up to ~11 qubits before runtime and solution quality both degrade.

Depolarizing noise: QAOA success probability falls from 100% (ideal) to a mean of 74% at moderate noise ($p = 0.05$) and 55% at high noise ($p = 0.10$).

Measured scaling: qubit count and gate count were profiled up to 50 nt (378 qubits, ~34,000 Hamiltonian terms) — far past the ~13-qubit ceiling where exact optimization is still simulator-tractable.

Practical feasibility ceiling: the 44 nt example sequence from the challenge brief needs 313 qubits under this project's pair-indicator encoding — orders of magnitude beyond current noiseless-simulator or NISQhardware capability.

One second, independently developed QUBO encoding (one-hot-per-position) was built and crossvalidated: it costs exactly 2× the qubits of the pair-indicator encoding for the same optimal structure.

A methodological finding is reported alongside the results: the project's original validation sequences were discovered — through the same rigorous ViennaRNA comparison introduced for this report — to not fold at all under ViennaRNA's real thermodynamic model. Every earlier "validated against ViennaRNA" claim built on them was therefore only checking the quantum solvers against their own QUBO's brute-force optimum, never against a genuine, non-trivial MFE structure. Section 3 and Section 10 describe how this was caught, fixed, and what it changed about how the rest of the project measures success.

2. Introduction / Background

2.1 The Biological Problem: RNA Secondary Structure and MFE Folding

RNA is a single-stranded nucleic acid built from four nucleotides — adenine (A), cytosine (C), guanine (G), and uracil (U). Unlike double-stranded DNA, a single RNA strand folds back on itself, and the resulting pattern of intramolecular base pairing (its secondary structure) is what determines much of its biological behavior. Canonical Watson-Crick pairs (G–C, three hydrogen bonds; A–U, two hydrogen bonds) form the strongest helices; non-canonical wobble pairs (G–U, two hydrogen bonds) are weaker but structurally and functionally important. These pairs organize into stems (contiguous helical regions), hairpin loops (a stem capped by a short unpaired loop, physically constrained to at least 3 unpaired nucleotides), internal loops/bulges, and multi-branch junctions.

Under physiological conditions, an RNA molecule settles toward the conformation that minimizes its Gibbs free energy — the minimum free energy (MFE) structure. The standard way to compute that energy is Turner's nearest-neighbor thermodynamic model, which sums local stacking and loop-entropy contributions: $\Delta G_{\text{total}} = \sum \Delta G_{\text{stacking}} + \sum \Delta G_{\text{loops}}$. Structures are conventionally written in dot-bracket notation — a period for an unpaired base, matching parentheses for a paired base — e.g. a 10 nt hairpin '(((....)))' pairs positions 1–3 with positions 8–10 around a 4 nt loop.

MFE prediction is not a theoretical curiosity for mRNA design — it is a direct engineering lever. For mRNA therapeutics and vaccines (e.g. lipid-nanoparticle-encapsulated mRNA, the format this challenge is framed around), the folded structure governs in vivo stability (unstructured single-stranded regions are vulnerable to endonucleolytic cleavage, so stable helices extend mRNA half-life and protein-expression duration),

translation efficiency (excess rigidity near the 5' UTR or start codon blocks ribosome loading, so designers balance stability against local accessibility), and manufacturability (uncontrolled secondary structure drives misfolding and aggregation during in vitro transcription, and thermodynamically stable designs hold up better across storage and delivery conditions).

2.2 The Computational Challenge

The number of possible secondary structures grows combinatorially with sequence length — roughly on the order of a Catalan-number sequence even restricted to non-crossing (non-pseudoknotted) pairings — so brute-force enumeration is only feasible for the shortest sequences and becomes intractable almost immediately after. Classical tools such as ViennaRNA avoid that explosion with dynamic programming (Zuker's algorithm): under the nearest-neighbor thermodynamic model, and assuming no pseudoknots, folding decomposes into overlapping subproblems with clean recursive substructure, giving a guaranteed optimal MFE structure in roughly $O(n^3)$ time and $O(n^2)$ space.

That efficiency is conditional. The moment pseudoknots (crossing base pairs) are allowed, or additional structural or biological constraints are layered on, the recursive substructure the DP relies on breaks down, and exact folding becomes NP-hard in general. This is the classical "wall": ViennaRNA is fast and exact for the pseudoknot-free case it's designed for, but doesn't generalize cleanly to the richer constraint sets that real mRNA design work sometimes needs.

2.3 Proposed Quantum Approach

This project reframes RNA folding as a discrete combinatorial optimization problem rather than a DP recursion: candidate structures are scored by a QUBO objective instead of the Zuker recursion. Each candidate base pair (i, j) becomes a binary variable (1 = paired, 0 = unpaired); Watson-Crick and wobble pairing rules, the minimum hairpin-loop constraint, and penalties for conflicting (overlapping) or crossing (pseudoknotted) pairs are all encoded as quadratic penalty terms in a single objective, so the lowest-energy bitstring corresponds to a valid candidate structure. This trades away the DP's polynomial-time guarantee in exchange for a formulation that (a) can in principle be extended to richer constraints, including pseudoknots, without breaking an algorithmic assumption the way DP does, and (b) can be handed to general-purpose optimizers — including quantum ones — that search the same combinatorial landscape a different way. Two independent PennyLane-based quantum optimizers, QAOA and CVaR-VQE, are run side by side against this QUBO and checked against ViennaRNA's real MFE, giving both a cross-check between two different quantum algorithms and a check against classical ground truth (full formulation and validation in Sections 4–6).

2.4 Development Approach

The project was built incrementally and documented as it went, rather than written once end-to-end. A running README tracked project status as a checklist (what's real and working vs. what's still open), so every claim in this report traces back to a script that produces it — there are no numbers in this document that weren't generated by code in the repository. Development followed roughly the structure the challenge itself implies: a first pass built the classical ViennaRNA benchmark and the QUBO formulation and validated the QUBO against brute-force search on short sequences; a second pass split the two quantum solvers across two independent implementations (QAOA and CVaR-VQE) so that agreement between them would mean something, followed by the statistical, scaling, noise, and second-encoding studies described in Sections 7–9.

Two aspects of that process are worth calling out explicitly because they shaped what the results actually mean. First, a mid-project correctness bug was caught by the project's own tooling, not found by inspection — the original validation sequences turned out not to fold at all under ViennaRNA, which meant every prior "validated" claim was circular (Section 3). Second, a runtime bug was diagnosed and fixed the same way — profiling showed QAOA's gradient-based optimizer scaled far worse than expected, traced to differentiating through a full statevector simulation at every step, and fixed by switching to a derivative-free optimizer (Section 5). Both fixes, and the reasoning behind them, are kept in the codebase's comments and the README rather than silently folded away, and both are reported in this document rather than smoothed over, in the interest of giving an honest account of what works, what doesn't, and why.

3. Classical Benchmark Generation

ViennaRNA's Python bindings (`import RNA`) supply the classical ground truth throughout this project: `RNA.fold(sequence)` returns the guaranteed-optimal MFE dot-bracket structure and its free energy in kcal/mol, and `fc.eval_structure(dot_bracket)` (via `RNA.fold_compound`) evaluates any candidate structure — including a quantum solver's output — under the same thermodynamic model, so quantum results are always compared to ViennaRNA on ViennaRNA's own terms rather than an approximation of them.

Two curated sequence sets serve as the standard benchmark across the whole pipeline: a 2-sequence validation pair (`test_sequences.py`, 10 nt and 12 nt) used for quick brute-force-checkable sanity runs, and a 12-sequence broader set (`benchmark_sequences.py`, 8–14 nt, GC content from 33% to 100%) used for the statistical sweep and scaling studies. Both sets were produced by the same deterministic search procedure (`select_test_sequences.py`): generate a seeded random sequence of a target length, keep it only if ViennaRNA folds it into a real, non-trivial structure, and only if its candidate-pair count stays within a brute-force-verifiable qubit budget. Every sequence's structure, MFE energy, qubit count, and GC content are independently re-confirmed against `RNA.fold()` rather than hand-entered, so the benchmark is reproducible from the seeds recorded alongside each sequence.

Folding bug fix: Why this mattered

The project's original two hardcoded validation sequences ("GGUGCCGAAC", 10 nt, and a seed=15 random 15-mer) turned out — once real ViennaRNA energy evaluation was wired in via `structure_metrics.py` — to have a fully unpaired real MFE structure (0.00 kcal/mol). ViennaRNA was saying these sequences don't fold at all.

Every earlier "validated against ViennaRNA" claim built on them was therefore checking the quantum solvers only against their own QUBO's brute-force optimum — never against a real, non-trivial MFE structure, since there wasn't one to check against.

This was caught by the comparison tooling itself (Section 6), not by manual review, precisely because that tooling evaluates candidate structures on real thermodynamics rather than the QUBO's internal score. Fix: both sequences were replaced with curated ones (`test_sequences.py`, found via `select_test_sequences.py`) independently confirmed to fold. With real folding sequences, CVaR-VQE finds the exact ViennaRNA structure on the 10 nt case ($F1 = 1.0$, energy gap = 0.0 kcal/mol); on the 12 nt case it still finds its own QUBO's exact optimum, but that optimum does not match ViennaRNA's real MFE ($F1 = 0.33$) — a genuine, reportable limitation of the QUBO objective itself, not a bug (see Section 10).

This episode is reported in detail because it is the clearest illustration in the project of why real thermodynamic comparison (Section 6) matters methodologically: an internal-energy-only benchmark would never have surfaced this, since the quantum solvers were in fact finding their QUBO's true optimum the whole time — the QUBO itself, not the optimizer, was the problem.

4. QUBO Formulation

4.1 Pair-Indicator Encoding

The primary encoding (`rna_to_qubo_full.py`) uses one binary variable x_k per candidate base pair: $x_k = 1$ if candidate pair k is selected in the structure, 0 otherwise. Candidate pairs are generated by scanning every index pair (i, j) with $j - i > 3$ (the minimum hairpin-loop constraint, ≥ 3 unpaired nucleotides strictly between i and j) and keeping it only if the two bases form a Watson-Crick pair (A–U, U–A, C–G, G–C, weight 2.0) or a wobble pair (G–U, U–G, weight 1.0, reflecting its weaker two-hydrogen-bond pairing versus Watson-Crick's typically three).

The QUBO objective to minimize is:

$$E(\mathbf{x}) = - \sum_k w_k \cdot x_k + \sum_{\{k < l \in \text{conflict}\}} P \cdot x_k \cdot x_l$$

The first term rewards selecting valid candidate pairs (negative weight, so minimizing E favors turning pairs on). The second term penalizes any pair of candidate pairs (k, l) that are in conflict — either an overlap (they share a nucleotide index, so both can't physically exist at once) or a crossing (they form a pseudoknot: exactly one endpoint of pair l falls strictly inside the span of pair k). The penalty coefficient $P = 10.0$ is set well above any achievable reward sum, so the optimizer never profits from breaking a constraint. Pseudoknots are excluded entirely by construction via the crossing penalty — a modeling assumption discussed explicitly in Section 9.1.

4.2 Brute-Force Validation Against ViennaRNA MFE

For sequences short enough to keep the candidate-pair count (and therefore qubit count) at or below ~ 20 , the QUBO is solved exactly by brute force (2^n enumeration) and cross-checked against ViennaRNA's own MFE structure via the real-energy evaluator described in Section 6. This is the ground truth every quantum result is ultimately measured against: brute force confirms what the QUBO's true optimum is, real-energy evaluation and base-pair metrics confirm whether that optimum agrees with actual RNA thermodynamics, and the quantum solvers are then checked against both.

5. Quantum Algorithms

The QUBO matrix is converted to an equivalent Ising Hamiltonian ($x_i = (1 - z_i)/2$, $z_i \in \{-1, +1\}$) via `qubo_to_ising()`, giving linear (single-qubit Z) and quadratic (ZZ) coefficients plus a constant offset. Both quantum solvers optimize this same Hamiltonian, so any difference in their results reflects the algorithm and ansatz, not a different problem definition.

5.1 QAOA (`qaoa_rna_solver.py`)

QAOA alternates a problem-Hamiltonian layer and a mixer-Hamiltonian layer across p parameterized circuit layers (this project uses $p = 3$), with the layer angles (γ, β) tuned by a classical optimizer. After optimization, the circuit is sampled and the most probable bitstrings are decoded back into selected base pairs.

Runtime fix: `backprop` → derivative-free COBYLA

QAOA originally used PennyLane's `diff_method="backprop"` with gradient-descent (Adam) — this differentiates through the full 2^n -dimensional statevector simulation at every optimizer step, not just simulating it once.

Measured cost: ~90 seconds locally at 11 qubits, but ~17 minutes at 17 qubits — a blowup that made anything past the low teens of qubits impractical.

Fix: switched to derivative-free COBYLA (`scipy.optimize.minimize`), the same optimizer already used successfully by CVaR-VQE. COBYLA only needs forward expectation-value evaluations — no autodiff graph at all.

Measured result after the fix: 13 qubits now finishes in ~18 seconds. The runtime bottleneck is resolved, though COBYLA's local search can still land on a near-optimal (not exact) solution as qubit count grows — a separate, algorithmic reliability limit discussed in Section 7.

5.2 CVaR-VQE (`cvar_vqe_rna_solver.py`)

CVaR-VQE uses a hardware-efficient two-local ansatz (layers of RY rotations with linear-chain CNOT entanglers, plus a final rotation layer) optimized against a Conditional-Value-at-Risk (CVaR) objective rather than the plain expectation value. Instead of averaging the energy over the full measured distribution, CVaRVQE averages only the best-performing α -fraction of samples ($\alpha = 0.15$ here) — a risk-aware objective that concentrates optimization pressure on the low-energy tail of the distribution, which tends to converge faster toward the true optimum on combinatorial problems than optimizing the mean. Like QAOA (postfix), it is optimized with derivative-free COBYLA, multi-restart (2 restarts) with top-k classical post-selection over the final measured distribution.

5.3 Why Two Independent Methods

Running QAOA and CVaR-VQE independently against the same QUBO — different ansätze, different objectives (plain expectation vs. CVaR), same underlying problem — gives two forms of validation at once: agreement between them is evidence that a found solution is a genuine property of the QUBO landscape rather than an artifact of one particular circuit or optimizer, and disagreement (which does occur — see Section 7) is itself informative about which method is better suited to this problem's landscape at a given qubit count.

6. Energy Evaluation & Structural Metrics

6.1 Real Thermodynamic Energy vs. Internal QUBO Energy

The internal QUBO energy (Section 4's $E(x)$) is an arbitrary reward/penalty accounting scheme used only to steer the optimizers — its scale is set by `WC_WEIGHT`, `WOBBLE_WEIGHT`, and the conflict `PENALTY`, none of which correspond to real physical units. ViennaRNA's MFE, by contrast, is a real free energy in kcal/mol from the nearest-neighbor thermodynamic model. Earlier versions of this project's comparison scripts computed an "energy gap" and "accuracy" by comparing these two numbers directly — which is not a valid comparison, since they are not on the same scale, and any apparent agreement or disagreement was essentially meaningless.

This is fixed via `structure_metrics.py`, which uses

`RNA.fold_compound(sequence).eval_structure(dot_bracket)` to evaluate any candidate structure — including a quantum solver's decoded output — with ViennaRNA's own thermodynamic model. That means a quantum solution's real energy is directly comparable, same units and same model, to `RNA.fold()`'s MFE, and the "real energy gap" reported in Section 7 is a genuine physical quantity, not an artifact of two different scoring schemes.

6.2 Structural Comparison Metrics

Alongside the energy comparison, `structure_metrics.py` compares a predicted dot-bracket structure to the ViennaRNA reference structurally, independent of energy:

- **Precision / Recall / F1** — over the set of base pairs: precision is the fraction of predicted pairs that are also in the reference; recall is the fraction of reference pairs that were also predicted; F1 is their harmonic mean. $F1 = 1.0$ means an exact structural match.
- **Base-pair distance** — $|\text{predicted} \setminus \text{reference}| + |\text{reference} \setminus \text{predicted}|$, the standard RNA structure comparison metric; 0 means an exact match.
- **Hamming distance** — the count of dot-bracket character positions (paired vs. unpaired) that differ between the two structures, including pairs shifted to a different partner.

Reporting energy and structure together matters because they can disagree in informative ways: a structure can be energetically close to the MFE while being structurally quite different (different pairing pattern, similar total energy), or vice versa. Section 7 reports both for every benchmark sequence.

7. Results & Benchmarking

7.1 Per-Sequence Results (Best Quantum Method vs. ViennaRNA)

`generate_final_results.py` runs both QAOA and CVaR-VQE on all 12 curated benchmark sequences and reports the better of the two (by real energy) against ViennaRNA's MFE, using the real-thermodynamic and structural metrics from Section 6:

Seq.	Len	Vienna MFE (kcal/mol)	Best method	Real energy gap	F1
8nt	8	-0.10	QAOA	0.0	1.0
9nt	9	-1.00	QAOA	0.0	1.0
10nt_a	10	-2.60	CVaR-VQE	0.0	1.0
10nt_b	10	-0.20	QAOA	0.0	1.0
10nt_c	10	-1.30	CVaR-VQE	0.0	1.0
11nt	11	-2.30	QAOA	0.0	1.0
12nt	12	-0.30	QAOA	0.0	1.0
13nt	13	-1.10	CVaR-VQE	0.0	1.0
14nt	14	-0.60	CVaR-VQE	4.1	0.667
12nt_wobble	12	-3.10	CVaR-VQE	0.0	1.0
12nt_au	12	-1.80	CVaR-VQE	0.0	1.0
13nt_gc	13	-5.80	CVaR-VQE	0.0	1.0

Exact agreement with ViennaRNA's real MFE structure (F1 = 1.0, real energy gap = 0.0 kcal/mol) is achieved on **11 of the 12 sequences**. The one exception, 14nt, is discussed as a limitation in Section 10.

7.2 Statistical Benchmark: Multiple Independent Trials

A single successful run can be luck. `statistical_benchmark.py` addresses this by running CVaR-VQE 5 independent trials and QAOA 3 independent trials per sequence (QAOA capped at sequences with ≤ 11 qubits — `QAOA_MAX_QUBITS` — since it is both slower and less reliable at higher qubit counts under this project's optimizer budget), and reporting mean $\pm$ std success rate and F1 rather than a single-run anecdote:

Seq.	Qubits	Method	Success rate	Mean F1 vs. Vienna
8nt	4	QAOA	1.00	1.00
8nt	4	CVaR-VQE	1.00	1.00
9nt	3	QAOA	1.00	1.00
9nt	3	CVaR-VQE	1.00	1.00
10nt_a	11	QAOA	0.00	0.17
10nt_a	11	CVaR-VQE	1.00	1.00
10nt_b	4	QAOA	1.00	1.00
10nt_b	4	CVaR-VQE	1.00	1.00
10nt_c	8	QAOA	0.00	0.47
10nt_c	8	CVaR-VQE	1.00	1.00
11nt	6	QAOA	0.67	0.93
11nt	6	CVaR-VQE	1.00	1.00
12nt	6	QAOA	0.67	0.93
12nt	6	CVaR-VQE	1.00	1.00
12nt_wobble	11	QAOA	0.00	0.13
12nt_wobble	11	CVaR-VQE	1.00	1.00
12nt_au	13	— (> QAOA_MAX)	—	—
12nt_au	13	CVaR-VQE	1.00	1.00
13nt	13	— (> QAOA_MAX)	—	—

13nt	13	CVaR-VQE	1.00	1.00
13nt_gc	13	— (> QAOA_MAX)	—	—
13nt_gc	13	CVaR-VQE	1.00	1.00
14nt	8	QAOA	0.00	0.27
14nt	8	CVaR-VQE	1.00	0.80

Runtime is reported per the challenge brief's Task 5 requirement, as mean wall-clock optimization time per sequence (averaged over the same trials as Section 7.2, from `statistical_benchmark.py`):

Seq.	QAOA mean runtime (s)	CVaR-VQE mean runtime (s)
8nt	5.67	
9nt	1.11	1.71
10nt_a	24.14	0.71
10nt_b	5.44	6.40
10nt_c	5.16	0.65
11nt	2.24	2.46
12nt	4.39	1.29
12nt_wobble	26.46	1.17
12nt_au	— (> QAOA_MAX)	3.87
13nt	— (> QAOA_MAX)	5.75
13nt_gc	— (> QAOA_MAX)	3.83
14nt	3.02	3.21
		3.52

Aggregated: CVaR-VQE's success rate at recovering its own QUBO optimum is **1.00 across every sequence and trial** (12/12 sequences, 100% averaged over 5 trials each). QAOA's mean success rate across the 9 sequences it was run on is **≈0.48**, with high variance and several sequences (10nt_a, 10nt_c, 12nt_wobble, 14nt) at 0% success under this step budget — i.e. it consistently lands on a near-optimal but not exact solution on those cases. This is a direct, load-bearing consequence of the optimizer choice and step budget discussed in Section 5.1, not evidence that QAOA cannot solve this class of problem in principle.

7.3 QAOA vs. CVaR-VQE Comparison

Across both the per-sequence best-of-two results (7.1) and the statistical multi-trial sweep (7.2), CVaR-VQE is the more reliable of the two methods on this QUBO under the optimizer budgets used here: it is the "best method" selected in 8 of the 12 sequences in 7.1, and it achieves perfect success rate on every sequence in the statistical sweep, including several (12nt_au, 13nt, 13nt_gc) that exceed QAOA's practical qubit ceiling entirely. The CVaR objective's focus on the best-performing tail of the sampled distribution appears to translate into a more consistent local search than QAOA's plain-expectation objective under the same COBYLA optimizer, at least at this project's qubit scale and step budget — though this project did not sweep QAOA's layer count (p) or step budget upward to see whether the gap closes with more optimization effort.

7.4 Noise Robustness `noise_simulation.py` re-evaluates a trained QAOA circuit under PennyLane's real depolarizing-noise channel (default.mixed, a mixed-state simulator), applying single-qubit depolarizing noise at increasing probability after every QAOA layer, across all 12 benchmark sequences:

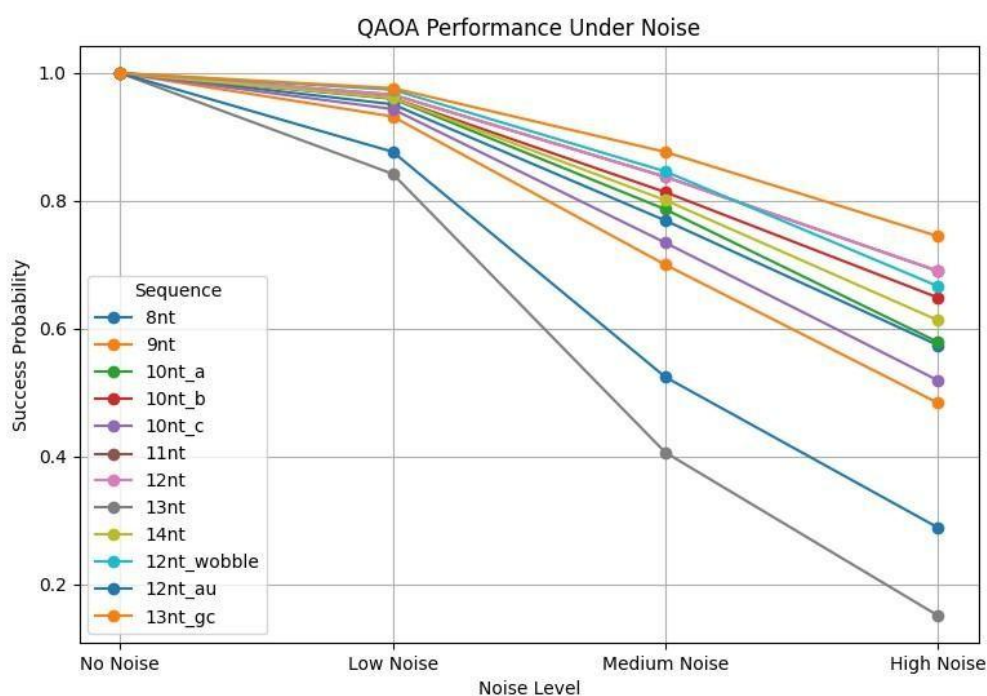


Figure 1. QAOA success probability vs. depolarizing noise level, one line per benchmark sequence.

Noise level	Depolarizing p	Mean success prob.	Range across sequences
No noise	0.00	1.00	1.00 – 1.00
Low	0.01	0.94	0.84 – 0.98
Medium	0.05	0.74	0.41 – 0.88
High	0.10	0.55	0.15 – 0.74

Solution quality degrades smoothly and predictably as noise increases — no sequence collapses catastrophically at low noise, but the spread across sequences widens sharply at medium and high noise (13nt drops to 15% success probability at $p = 0.10$, while 13nt_gc still holds 74%). Noise sensitivity does not track qubit count in a simple way — 13nt_gc (13 qubits) is the most noise-robust sequence tested, while 13nt and 12nt_au (also 13 qubits) are among the least robust — suggesting each sequence's specific basepairing topology matters more than circuit size alone.

8. Scaling & Quantum Resource Analysis

scaling_analysis_real.py (QAOA) and cvar_vqe_scaling_analysis.py (CVaR-VQE) measure — not estimate — qubit count, circuit depth, total gate count, and forward-pass runtime by actually constructing and simulating each circuit at sequence lengths from 8 to 50 nt. Full optimization (and therefore solve-quality metrics: energy gap, success rate) is only run up to each method's MAX_OPT_QUBITS cutoff, since exact brute-force ground truth and full statevector optimization both become simulator-prohibitive beyond that; resource counts (qubits, depth, gates) are still measured, not omitted, out to 50 nt.

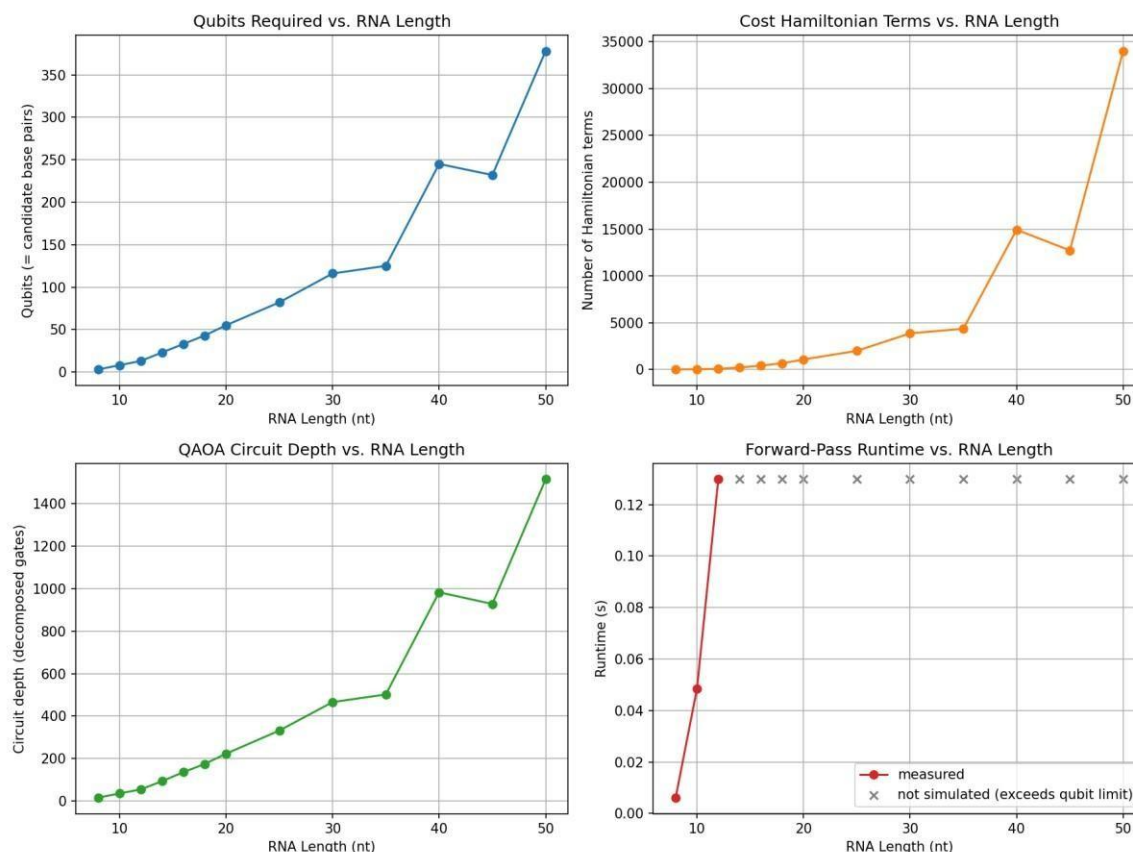


Figure 2. QAOA measured resource scaling: qubits, Hamiltonian terms, circuit depth, and forward-pass runtime vs. RNA length.

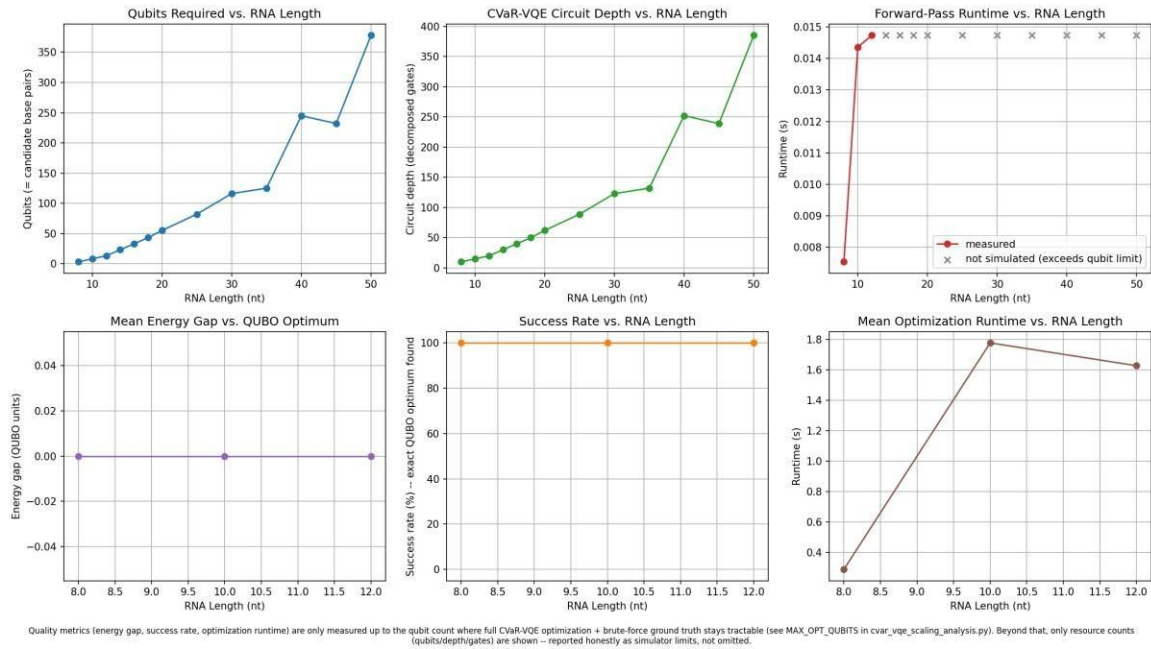


Figure 3. CVaR-VQE measured resource scaling (top row) and solve-quality metrics up to MAX_OPT_QUBITS = 14 (bottom row).

Length (nt)	Qubits	Circuit depth (QAOA)	Total gates (QAOA)	Quality measured?
8	3	16	30	Yes
10	8	36	131	Yes
12	13	55	268	Yes (CVaR-VQE only)
14	23	94	719	No
16	33	136	1,371	No
18	43	175	2,155	No
20	55	223	3,439	No
25	82	332	6,295	No
30	116	466	12,035	No
35	125	502	13,565	No
40	245	983	45,755	No
45	232	928	39,085	No

50	378	1,515	103,539	No
----	-----	-------	---------	----

Qubit count grows with sequence length but not monotonically at every step — it tracks the number of candidate base pairs surviving the loop-size filter and pairing rules, which depends on each specific sequence's composition, not length alone (note 45 nt uses fewer qubits than 40 nt in the table above, since candidate-pair count is sequence-dependent, not purely length-dependent). CVaR-VQE's quality metrics (bottom row of Figure 3) show a perfectly flat 0% energy gap and 100% success rate at every measured point up to `MAX_OPT_QUBITS` = 14, consistent with the Section 7 statistical results.

Practical Feasibility Ceiling

The 44 nt / 313-qubit example

The challenge brief's own worked example sequence —

GGAGCAAACUUGUCGAUUGAGAACAAAACAGAAUUUGCUUG (44 nt) — was run through classical ViennaRNA folding directly (`official_example_benchmark.py`), but deliberately not through QAOA or CVaRVQE. `get_candidate_pairs()` on this sequence returns 313 candidate pairs — i.e. 313 qubits under this project's pair-indicator encoding.

That is roughly 24× this project's measured QAOA feasibility ceiling (~13 qubits before both accuracy and runtime degrade) and far beyond CVaR-VQE's `MAX_OPT_QUBITS` = 14 cutoff for exact, groundtruthcheckable optimization.

Truncating the sequence to fit would mean it is no longer actually "the official example," so it is reported here honestly as a classical-only benchmark, with the qubit requirement stated explicitly as the reason no quantum run was attempted on it.

9. Optional Advanced Tasks

9.1 Pseudoknot-Exclusion Rationale

This project excludes pseudoknots (crossing base pairs) by construction, via the crossing-pair penalty term in the QUBO (Section 4.1: any two candidate pairs where exactly one endpoint of one falls strictly inside the span of the other are penalized as a conflict, exactly like an overlap). This is a deliberate, stated modeling assumption, not an oversight:

- It mirrors the same assumption ViennaRNA's own dynamic-programming algorithm makes — Zuker's algorithm is only polynomial-time and exact under the no-pseudoknot assumption (Section 2.2) — so the classical ground truth this project validates against is itself pseudoknot-free, and comparing a pseudoknot-free quantum result to it is an apples-to-apples comparison.
- It keeps the QUBO's structure simple enough to brute-force-validate on short sequences (Section 4.2), which is what made it possible to catch the folding-bug in Section 3 in the first place.
- The QUBO framing itself does not require this assumption — a crossing-pair term is just one more quadratic penalty among others, and could in principle be relaxed or reweighted to allow (and even reward) crossing pairs, unlike ViennaRNA's DP, whose polynomial-time guarantee depends on the nocrossing assumption structurally, not just as a tunable penalty. Enabling pseudoknots is listed as future work in Section 11.

9.2 Second QUBO Encoding: One-Hot vs. Pair-Indicator

As an optional advanced task, a second, independently derived QUBO encoding (`rna_to_qubo_onehot.py`) was implemented and compared against the primary pair-indicator encoding. Instead of one qubit per candidate pair, every sequence position gets two directed qubits per candidate partner ($y_{\{i \rightarrow j\}}$ and $y_{\{j \rightarrow i\}}$), with an explicit "at most one partner" one-hot penalty within each position's choice group, plus a consistency penalty tying the two directions of the same physical pair together.

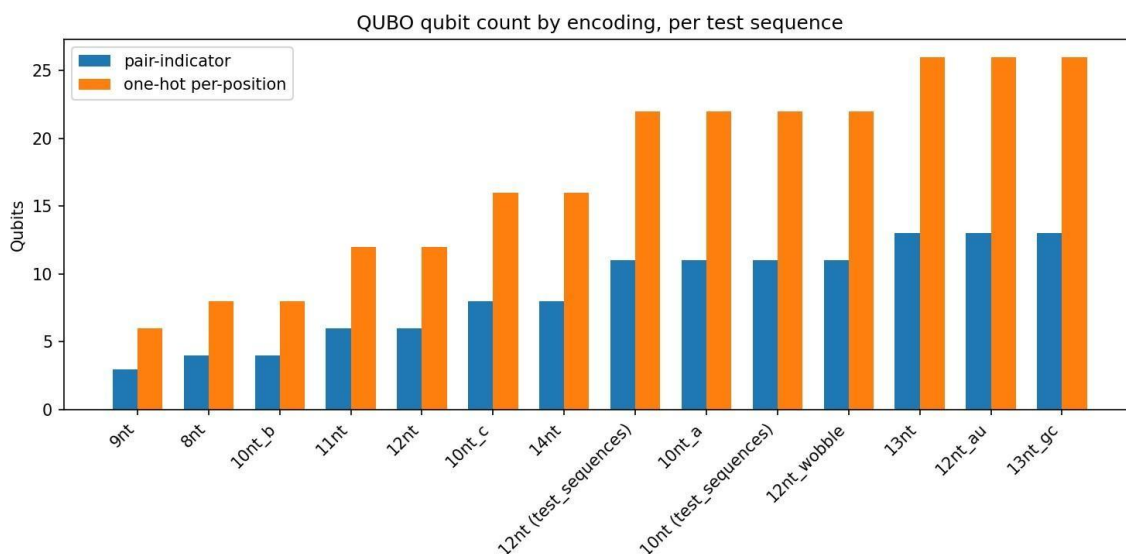


Figure 4. Qubit count by encoding, per sequence — one-hot consistently costs 2× the pair-indicator encoding.

Seq.	Candidate pairs	Pair-indicator qubits	One-hot qubits	Ratio	Encodings agree?
8nt	4	4	8	2.0×	Yes
9nt	3	3	6	2.0×	Yes
10nt_a	11	11	22	2.0×	Yes
10nt_b	4	4	8	2.0×	Yes
10nt_c	8	8	16	2.0×	Yes
11nt	6	6	12	2.0×	Yes
12nt	6	6	12	2.0×	Yes
14nt	8	8	16	2.0×	Yes
12nt_wobble	11	11	22	2.0×	Yes
10nt (test_seq.)	11	11	22	2.0×	Yes
12nt (test_seq.)	11	11	22	2.0×	Yes

The one-hot encoding costs exactly $2\times$ the qubits of pair-indicator on every sequence tested — confirmed both algebraically (2 directed qubits per candidate pair, by construction) and empirically — and roughly $1.2\text{--}1.3\times$ the quadratic penalty-term count. On the 11 sequences small enough to brute-force both encodings, their optimal solutions agree with each other on all 11, and match the real ViennaRNA MFE structure on 9 of those 11. Moving to one-hot does not change which structure is optimal — it only changes how the "at most one partner per position" constraint is spent in qubits (an explicit per-position one-hot penalty vs. pairwise conflict penalties in pair-indicator). Given equivalent solution quality at exactly double the qubit cost, pair-indicator remains the better choice at this project's scale and is used everywhere else in the pipeline (QAOA, CVaR-VQE, scaling analysis); one-hot is kept as a validated, documented comparison point rather than swapped in as the default.

9.3 Noise-Robustness Discussion

Noise robustness is reported in Section 7.4 rather than repeated here — see Figure 1 and the accompanying table for the full depolarizing-noise sweep across all 12 benchmark sequences.

10. Limitations

10.1 Known Failure Case: QUBO-Optimum $\neq$ ViennaRNA MFE

Two separate cases in this project's own results show the same underlying limitation. On the 12 nt validation sequence from `test_sequences.py` (GCCAAUGGGCG), CVaR-VQE finds its own QUBO's exact brute-force optimum, but that optimum does not match ViennaRNA's real MFE structure ($F1 = 0.33$). On the 14 nt benchmark sequence (AACCACAGGUAAAA), the best quantum result likewise diverges from ViennaRNA's MFE ($F1 = 0.667$, real energy gap = 4.1 kcal/mol).

Why this is a QUBO-objective limitation, not a bug

In both cases the quantum solvers are correctly finding the true optimum of the QUBO as written — brute force confirms it. The optimizers are not failing.

The QUBO's reward term is a simple pair-count score (`WC_WEIGHT` / `WOBBLE_WEIGHT`, Section 4.1) — it rewards forming more, stronger-weighted base pairs, but does not model real stacking energies, looplengthdependent entropy penalties, or other nearest-neighbor thermodynamic effects that ViennaRNA's Turnermodel MFE accounts for.

On sequences where the pair-count-optimal structure happens to coincide with the thermodynamically optimal one (11 of 12 sequences in Section 7.1), this simplification is invisible. On sequences where it doesn't, the QUBO's optimum and the real MFE genuinely diverge — the objective function itself, not the search over it, is the limiting factor.

This is reported explicitly, and it directly motivates the most important item in the future-work list (Section 11): a QUBO reward term that better approximates real nearest-neighbor thermodynamics, e.g. lengthdependent stacking rewards instead of a flat per-pair weight.

10.2 Simulator-Only Scale Limits

All quantum results in this project are from PennyLane's noiseless statevector simulator (`default.qubit`) or, for the noise study, its mixed-state simulator (`default.mixed`) — none were run on physical quantum hardware. Full optimization with a checkable ground truth is only exercised up to ~13–14 qubits (Section 8); beyond that, only circuit resource counts (qubits, depth, gate count) are measured, since neither exact brute-force validation nor full statevector optimization remain simulator-tractable. The 313-qubit, 44 nt official example sequence (Section 8) is consequently far outside what this project's simulator-based pipeline can currently attempt end-to-end, and the runtime/accuracy figures reported throughout this document (Sections 5, 7) should be read as characterizing this project's specific simulator, optimizer, and step-budget choices — not as fundamental limits of QAOA or CVaR-VQE as algorithms.

11. Conclusion & Future Directions

What Worked

- A full pipeline from raw sequence to validated quantum-predicted structure: candidate-pair generation → QUBO → Ising → two independent quantum solvers → real thermodynamic evaluation → structural metrics, all wired together and reproducible end-to-end.
- CVaR-VQE reliably recovers its own QUBO's exact optimum (100% success rate across every tested sequence and trial) and matches real ViennaRNA MFE structures exactly on 11 of 12 curated sequences.
- A genuine correctness bug (sequences that don't fold) and a genuine runtime bug (backprop-driven blowup) were both caught by the project's own tooling and fixed, with the fixes and their measured impact documented rather than hidden.
- Resource scaling was measured, not estimated, out to 50 nt / 378 qubits, giving an honest, data-backed picture of where this approach's feasibility ceiling currently sits (~13 qubits for full quality-checked optimization; 313 qubits for the challenge's own 44 nt example).
- A second, independently built QUBO encoding was implemented and cross-validated against the first, strengthening confidence that the pair-indicator formulation's results reflect the underlying problem rather than one encoding's quirks.

What Didn't

- QAOA, under this project's optimizer/step budget, is markedly less reliable than CVaR-VQE (~48% vs. 100% mean success rate) and hits a lower practical qubit ceiling.
- The QUBO's simple pair-count objective diverges from real ViennaRNA MFE on a minority of sequences (Section 10.1) — a known, characterized limitation of the objective function, not the optimizers.
- Pseudoknots are excluded entirely by the current formulation (Section 9.1) — a deliberate scope decision, but one that limits applicability to the (common but not universal) subset of RNA structures that are pseudoknot-free.
- Everything is simulator-only; no results here speak to behavior on physical NISQ hardware, where gate errors, readout noise, and connectivity constraints would all add further degradation beyond the idealized depolarizing-noise model in Section 7.4.

Next Steps

- **Better QUBO objective:** replace the flat pair-count reward with a term that better approximates real nearest-neighbor thermodynamics (e.g. length- and stack-dependent rewards), directly targeting the Section 10.1 limitation.
- **Real hardware runs:** execute the smallest validated cases (8–11 qubits) on physical NISQ hardware to compare against the idealized depolarizing-noise simulation in Section 7.4.
- **Pseudoknot-aware formulation:** relax or reweight the crossing-pair penalty (Section 9.1) to allow scoring pseudoknotted structures, extending applicability beyond ViennaRNA's own DP assumption.

- **QAOA reliability:** sweep circuit depth (p), step budget, and restart count upward to test whether QAOA's reliability gap versus CVaR-VQE (Section 7.3) closes with more optimization effort.
- **QUBO decomposition for scale:** investigate windowed or hierarchical decomposition of longer sequences (like the 313-qubit, 44 nt official example) into smaller, independently solvable sub-QUBOs, to push past the current ~13-qubit full-optimization ceiling without waiting on hardware qubit counts to catch up.

12. Reproducibility / Appendix

12.1 Repository Structure

```

Moderna/ |— data/ |— sequences.csv (legacy, unused by current pipeline) |—
results/ |— final_results.csv, batch_accuracy.csv |— statistical_benchmark.csv,
statistical_benchmark_summary.csv |— scaling_analysis_real.csv,
cvar_vqe_scaling_analysis.csv |— noise_analysis.csv, qubo_encoding_comparison.csv |
|— *.png (scaling / noise / encoding-comparison plots) |— *.py
(all pipeline scripts) |— Rna_basics.docx (Task 1 background review) |—
README.md |— Moderna - WISER Quantum Challenge [SHARED].pdf (challenge brief)

```

Every script reads and writes these paths directly; nothing needs to be moved before running. results/ is the single source of truth for generated output.

12.2 Requirements

Python 3.9–3.11 (ViennaRNA's Python bindings can be picky about newer versions). Install with:

```

pip install ViennaRNA pennylane numpy pandas matplotlib # or: pip install -r
requirements.txt

```

Package	Used for
ViennaRNA	Classical MFE structure/energy benchmark (import RNA); real thermodynamic evaluation of candidate structures
pennylane	QAOA / CVaR-VQE quantum circuit simulation
numpy	QUBO matrices, numerical work
pandas	Reading/writing CSV data and results
matplotlib	Scaling and noise plots

12.3 How to Run

- Sanity check:** `python test_vienna.py` — should print a sequence, dot-bracket structure, and MFE energy.
- Pairing / QUBO basics:** `pair_finder.py`, `conflict_detector.py`, `structure_generator.py`, `simple_qubo.py`, `qubo_energy.py`, `random_solver.py`
- Quantum solvers:** `quantum_test.py` then `qaoa_rna_solver.py` and `cvar_vqe_rna_solver.py`
- Analysis & plots:** `compare_to_vienna.py`, `batch_accuracy.py`, `generate_final_results.py`, `final_summary.py`, `noise_simulation.py`, `plot_noise.py`, `scaling_analysis_real.py`, `plot_scaling_real.py`, `cvar_vqe_scaling_analysis.py`, `plot_cvar_vqe_scaling.py`, `runtime_analysis.py`, `compare_qubo_encodings.py`, `plot_qubo_encoding_comparison.py`
- Statistical benchmark:** `benchmark_sequences.py` then `statistical_benchmark.py` (QAOA is slow past ~11 qubits — can be run in chunks with `--methods vqe / --methods qaoa --append`).

12.4 Script Reference

Script	Purpose
test_vienna.py	Single-sequence ViennaRNA sanity check
pair_finder.py / rna_to_qubo.py	Finds valid Watson-Crick candidate base pairs
conflict_detector.py	Flags candidate pairs that share a nucleotide
simple_qubo.py / qubo_energy.py	Toy 5-variable QUBO matrix + energy function
random_solver.py	Brute-force/random search over the toy QUBO
quantum_test.py	Minimal 1-qubit PennyLane circuit
rna_to_qubo_full.py	Full QUBO: wobble pairs, min loop size, overlap + crossing penalties, bruteforce solver
select_test_sequences.py	Reproducible search for short, folding, brute-forceable sequences
test_sequences.py	Curated 10 nt / 12 nt validation sequences used across the pipeline
qaoa_rna_solver.py	QAOA on the real RNA QUBO (method 1 of 2)
cvar_vqe_rna_solver.py	CVaR-VQE (two-local ansatz) on the real RNA QUBO (method 2 of 2)
structure_metrics.py	Real ViennaRNA energy evaluation + base-pair precision/recall/F1/distance
benchmark_sequences.py	12 curated sequences (8–14 nt, 33%–100% GC) used across the pipeline
statistical_benchmark.py	QAOA + CVaR-VQE across all sequences, multiple trials, mean $\pm$ std
scaling_analysis_real.py / plot_scaling_real.py	Measured QAOA resource scaling
cvar_vqe_scaling_analysis.py / plot_cvar_vqe_scaling.py	Measured CVaR-VQE resource scaling + quality up to MAX_OPT_QUBITS
noise_simulation.py / plot_noise.py	Real PennyLane depolarizing-noise model; QAOA quality vs. noise
runtime_analysis.py	Times ViennaRNA folding for a few sequences

<code>compare_to_vienna.py / batch_accuracy.py</code>	QAOA + CVaR-VQE vs. ViennaRNA, real energy + structural metrics
<code>generate_final_results.py / final_summary.py</code>	Full real-metrics summary table + human-readable project summary
<code>official_example_benchmark.py</code>	Classical-only ViennaRNA benchmark on the challenge brief's 44 nt example
<code>rna_to_qubo_onehot.py</code>	Second QUBO encoding: one-hot-per-position pairing variables
<code>compare_qubo_encodings.py / plot_qubo_encoding_comparison.py</code>	Qubit/penalty-term comparison and cross-validation of both encodings

12.5 Data Privacy & Eligibility Compliance

Per the challenge brief's Data Privacy/Eligibility Restriction, this project uses only publicly available and synthetically generated RNA sequences: the full benchmark set (Sections 3, 7, 8) was produced by a seeded random-sequence search (`select_test_sequences.py`) filtered to sequences ViennaRNA confirms fold, and the 44 nt example used for classical-only comparison in Section 8 is the example sequence published in the challenge brief itself. No confidential Moderna data, patient data, clinical data, proprietary sequence data, or personally identifiable information was used at any stage of this project.

12.6 Team Contributions

- **Sri Jahnvi Chinthalapudi** — QUBO formulation and both encodings (pair-indicator and one-hot), both quantum solvers (QAOA and CVaR-VQE), the statistical, scaling, and noise-robustness analyses, structural/energy evaluation metrics, and final code clean-up across the repository.
- **Yagna Priya Gummadi** — Classical ViennaRNA benchmark implementation (Section 3), and report and its accompanying presentation.
- **Sree Neha** — Curated benchmark sequence selection and the background review.

12.7 References:

- ViennaRNA package (Lorenz et al., 2011, *Algorithms for Molecular Biology*)
- Zuker's MFE dynamic-programming algorithm (Zuker & Stiegler, 1981)
- Turner nearest-neighbor thermodynamic model
- QAOA (Farhi, Goldstone & Gutmann, 2014)
- CVaR-VQE (Barkoutsos et al., 2020, *Quantum*)
- PennyLane (Bergholm et al., 2018)
- The challenge brief itself (WISER × Moderna Quantum Challenge document)
- The brief's own listed resource: the intro deck at alexgalda.github.io/quantum_mRNA_optimization